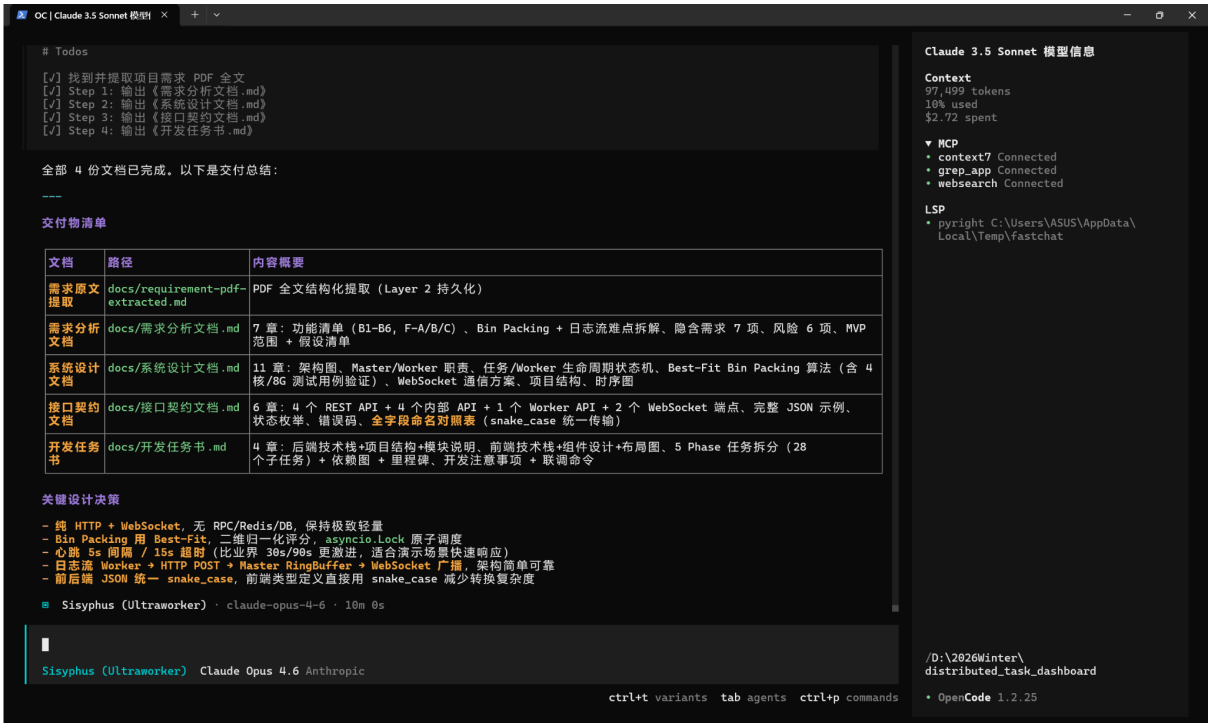
输入：
请查看根目录的项目需求 PDF。请严格遵守“先设计，后编码”的原则，不要立刻写任何代码。
技术栈要求：
后端：Python + FastAPI
- 前端：Vue 3 (Composition API) + Typescript
你的任务是充当架构师，为项目建立完善的文档规范。请一步步执行以下 4 个步骤，每完成一部分，就将其保存到你创建的 docs/ 文件夹下，然后再进行下一步：
第一步：输出《需求分析文档.md》
明确：必做功能、难点（特别是要求中的 Bin Packing 算法 和日志实时推流）、隐含要求、风险点、最小可交付版本（MVP）范围。
第二步：输出《系统设计文档.md》
明确：系统总体架构、Master / Worker 职责划分、前后端职责划分、任务与 Worker 的生命周期、调度策略（明确说明如何处理文档中的 2核/4G 测试用例）、实时通信方案。
第三步：输出《接口契约文档.md》
明确：REST API 列表 、请求/响应 JSON 示例、WebSocket 消息数据帧格式 、状态枚举（Pending/Running/Success/Failed/OFFLINE等）、错误码、统一的字段命名规范（必须明确前后端交互时的命名风格）。
第四步：输出《开发任务书.md》基于以上设计，输出：前端开发文档、后端开发文档、开发优先级与任务拆分建议。全局要求：所有文档必须彼此一致，字段定义不能有任何省略。对不确定的地方要明确说明假设。优先保证可实现性，不要过度设计。

Agent输出：



输入：
我需要你修改 docs/ 目录下的相关文档（主要是 开发任务书.md 和 系统设计文档.md），将后端的项目结构彻底重构。

修改要求：

请根据根目录 AGENTS.md 文件中 1.1 Recommended Directory Layout 的标准，重新规划后端的目录结构。

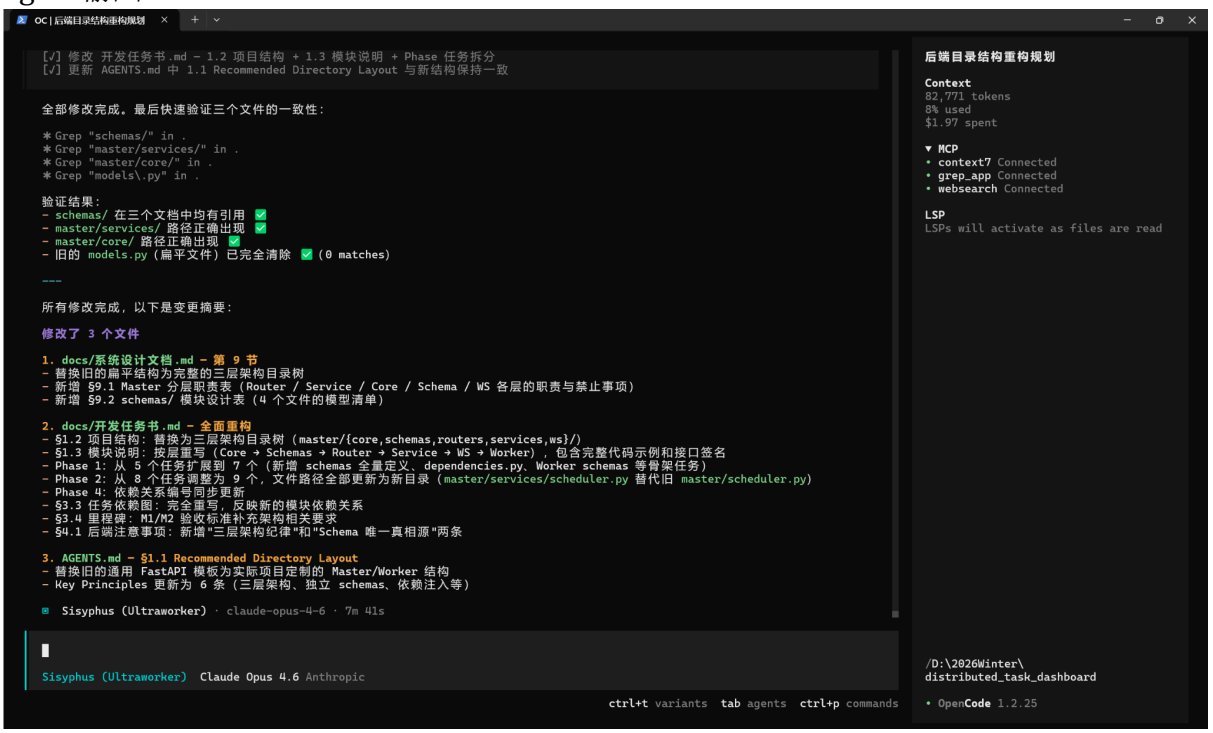
Master 必须严格保留企业级的三层架构（Router / Service / Core 层）。

Worker 按需设计结构

必须设立独立的 schemas/ 文件夹（或独立的 schema 模块），以此来严格规范和保证 REST API 和 WebSocket 请求体和响应体字段的一致性。

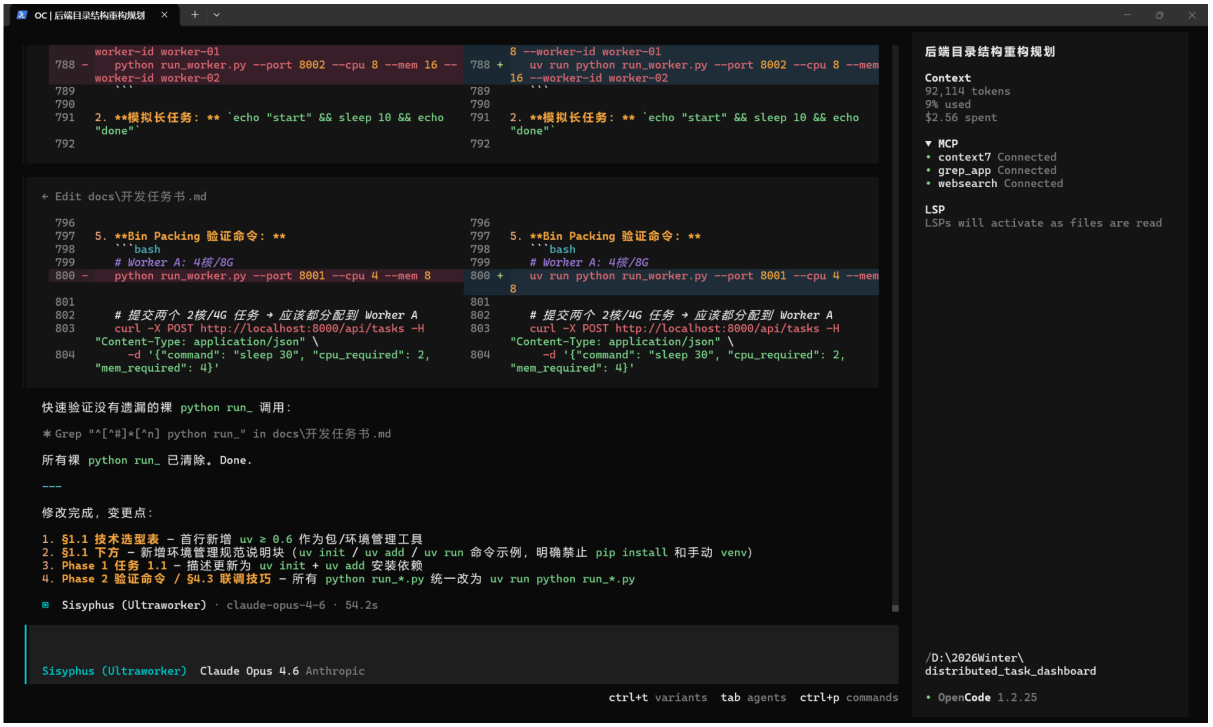
注意：前端架构、WebSocket 设计以及《接口契约文档.md》中的内容保持不变。

Agent输出：



输入：
后端需要使用uv管理虚拟环境，在后端说明文档里修改并加上这个要求

Agent输出：



输入:
ulw 请读取更新后的 `docs/` 目录下的《系统设计文档.md》和《开发任务书.md》。确认你已经完全掌握了重构后的“控制面与数据面分离”的微服务架构后，我们正式开始全栈代码编写。

请严格按照文档，同步推进并完成 **Phase 1** 到 **Phase 4** 的所有前后端开发工作。

【后端开发强约束】

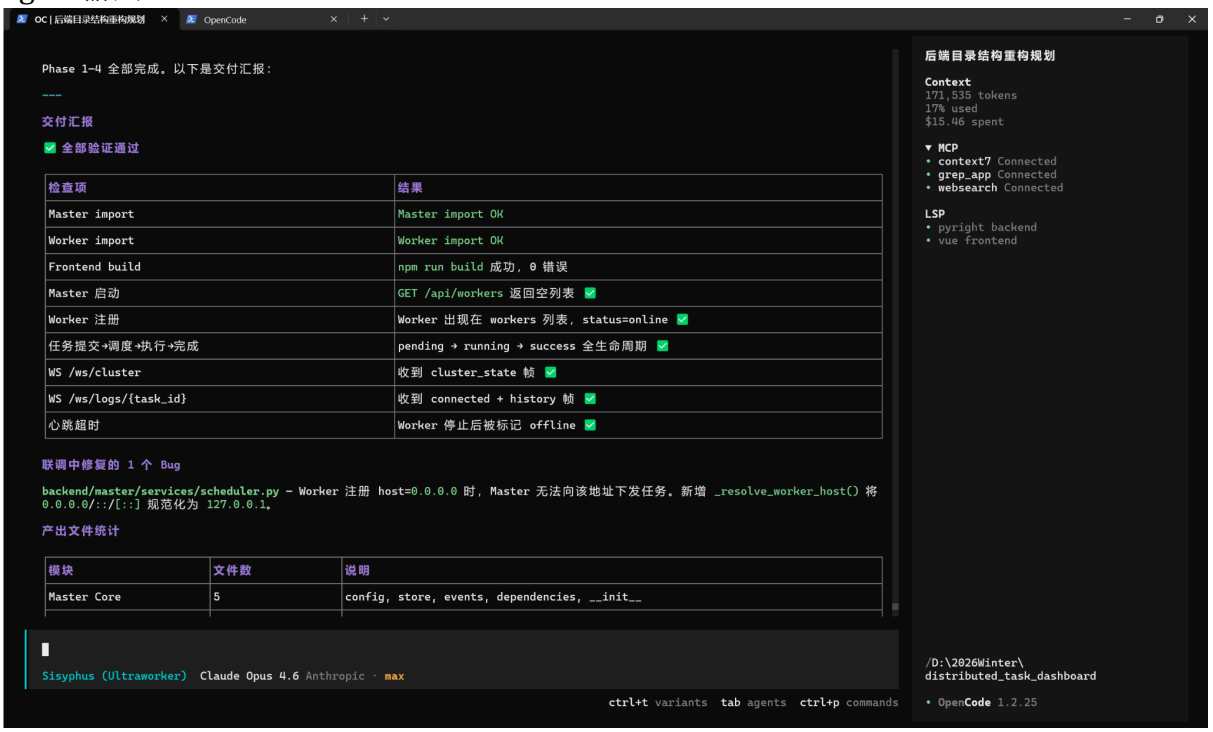
1. **Master** 节点(控制面):必须严格按照开发文档进行开发
   - 调度逻辑(Bin Packing)必须封装在 Service 层。
   - 心跳监控机制:必须在 Core 或 Service 层实现后台异步任务(`asyncio` 循环），严格执行 5 秒扫描、15 秒超时的逻辑。若 Worker 超时，必须标记为 OFFLINE，将相关 Running 任务置为 Failed，释放资源并尝试重新调度。
   - 所有的 REST API 请求和心跳，响应必须通过 `schemas/` 层的 Pydantic 模型进行严格校验。
2. **Worker** 节点(数据面):保持扁平化结构，严禁使用三层架构。
   - 心跳上报机制:必须实现一个后台常驻协程，严格每 5 秒向 Master 发送一次 HTTP 心跳。
   - 必须创建独立的 `schemas` 模块，用于强校验与 Master 通信(如心跳、注册、日志)的所有 Payload。
3. 字段契约:所有 JSON 序列化字段必须严格遵守《接口契约文档.md》中的 `snake_case` 规范。

【前端开发强约束与特殊面试指令】

1. 整体 **UI** 风格：项目需求明确要求前端必须“炫酷”。请使用 Vue 3 + Element Plus (或 Tailwind CSS) 构建一个极具科技感的暗黑主题（Dark Mode）监控大盘（Dashboard）。
2. 集群热力图 **(Cluster Heatmap)**：Worker 卡片必须有高级感。资源占用率请使用平滑的动画进度条或仪表盘，状态改变（如变成 OFFLINE）时必须有显眼的过渡动画。
3. 日志模块特殊指令 **(LogModal)**：
    - 当前阶段，对于日志流模态框，你只需搭建炫酷的 **UI** 外壳和最基础的 **WebSocket** 数据接收展示即可。
    - 绝对不要在现在去实现“自动滚动到底部”的逻辑！
    - 绝对不要做任何防卡死（节流/防抖/虚拟滚动）的性能优化！
    - 核心逻辑和性能瓶颈我们留在 Phase 5 单独处理。

请在完成上述代码编写后向我汇报

Agent输出：



输入：
帮我解决一下上传任务mem单位是mb但是后端理解的是Gb的问题

Agent输出：

输入：
我需要你指派前端agent重构网页和重新设计UI/UX，前端功能正常不需要做修改，以下是对前端页面的要求：

UI/UX 设计，现代硬核极客风数据看板（Dashboard），长网页滚动布局。

整体色彩：严格的深色模式，极致克制。放弃所有高饱和度霓虹色点缀，仅使用深炭灰、中灰和纯白色来构建层次，视觉风格冷峻、专业，类似 Vercel 或 Linear 的高级感。

布局结构：

1. 左侧边栏：高度优化的固定侧边栏，包含可折叠的层级菜单和极简的单色线框图标。
2. 主内容区：向下无限延伸的滚动设计。采用不对称或多尺寸的卡片网格布局。

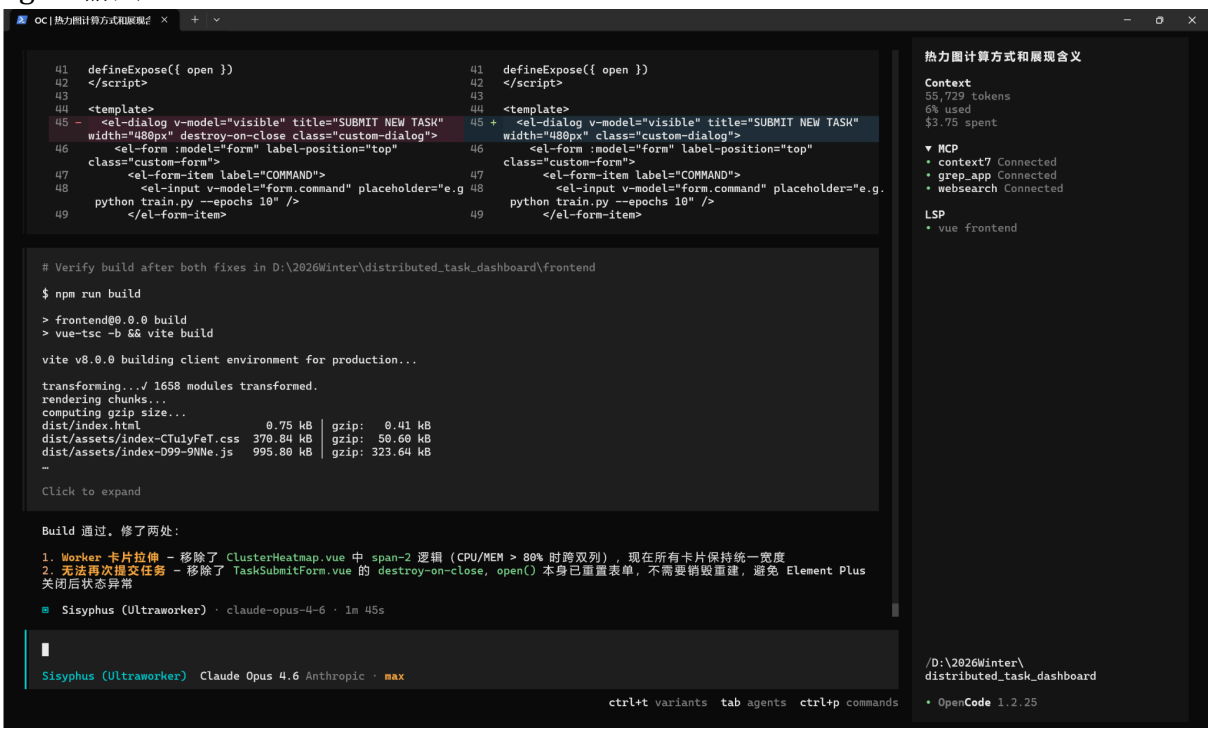
核心卡片与交互展示：画面中需展示多张极深灰色背景的极简数据卡片。重点：画面中心展示一张“被点击后展开”的大卡片，内部呈现一个高度直观的数据热力图（**Heatmap**）。热力图同样使用低饱和度的单色阶梯来展示数据密度。

高级感来源：不依赖华丽色彩，而是通过极致的数据排版（等宽数字）、留白、精细的 1px 边框和清晰的逻辑层级来展现产品的专业性和功能层面的强大。网页长图展示。

Agent输出：

输入：有一个问题是当我创建了一个任务过后worker卡片会在x轴方向上延申填充，需要修复一下这个问题，而且我创建了一个任务过后就不能创建第二个任务了

Agent输出：

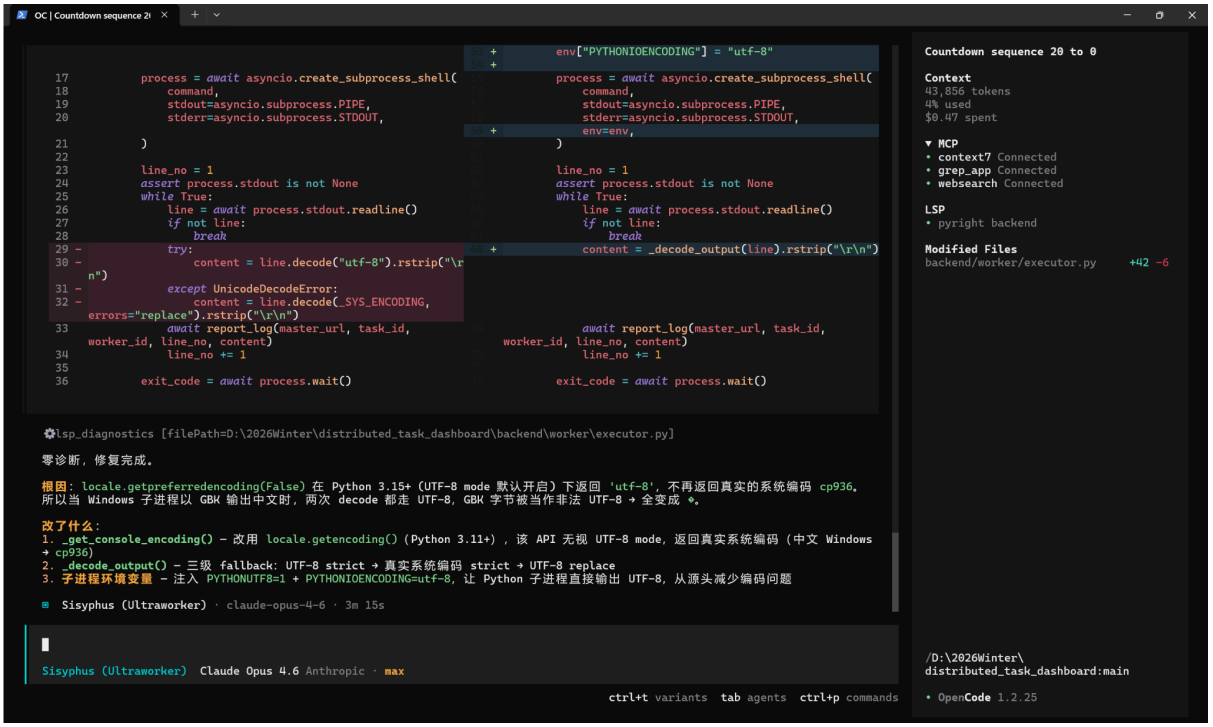


输入：

接下来是一些风格上需要打磨的地方。热力图的颜色和worker卡片中进度条的颜色可以换成饱和度高一点的颜色，可视化的效果会更好。最近任务框点击刷新没有动画效果。recent task 框中的workerID使用的是id，而卡片使用的是名字导致对应不起来。connected字符和submit功能在sidebar和界面顶部同时出现了，请只保留顶部的部分。submit new task框的command框的y轴宽度可以更宽一点，输入的command字符使用高亮极客风展示高级的效果。热力图x轴的worker没有对应编号导致用户对应不上。viewlogs展示的日志读出来的是乱码。

Agent输出：

输入：
]̣̣̣



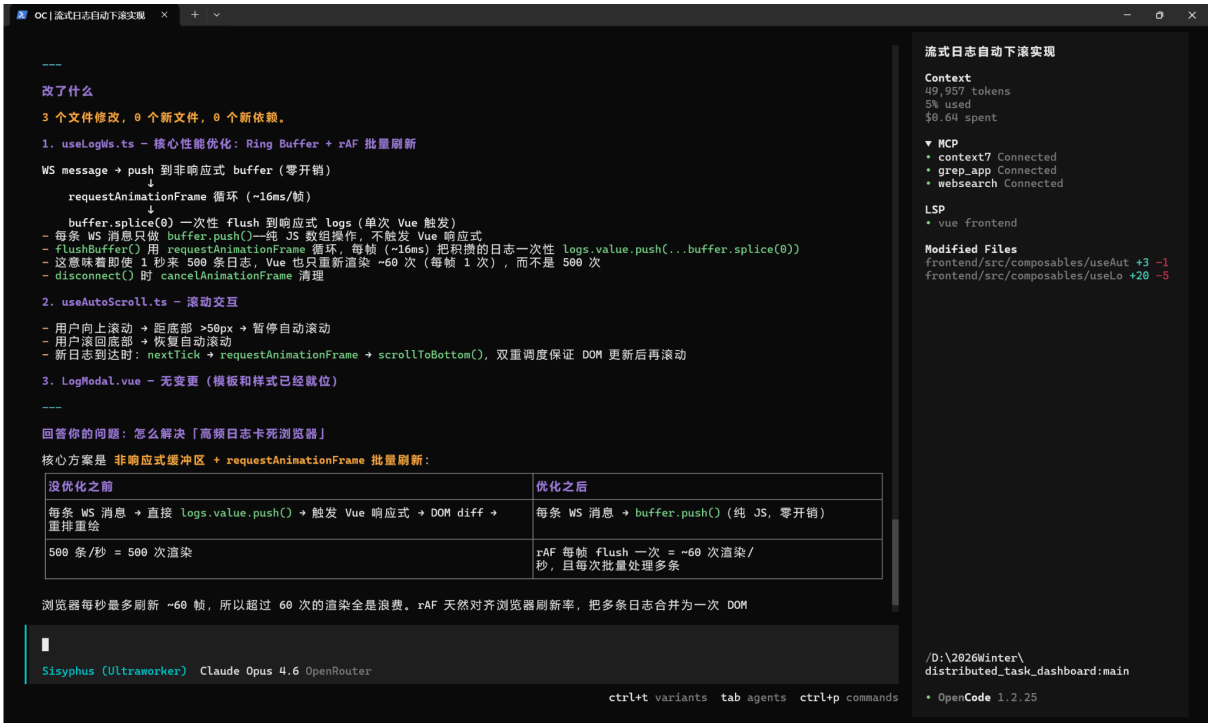
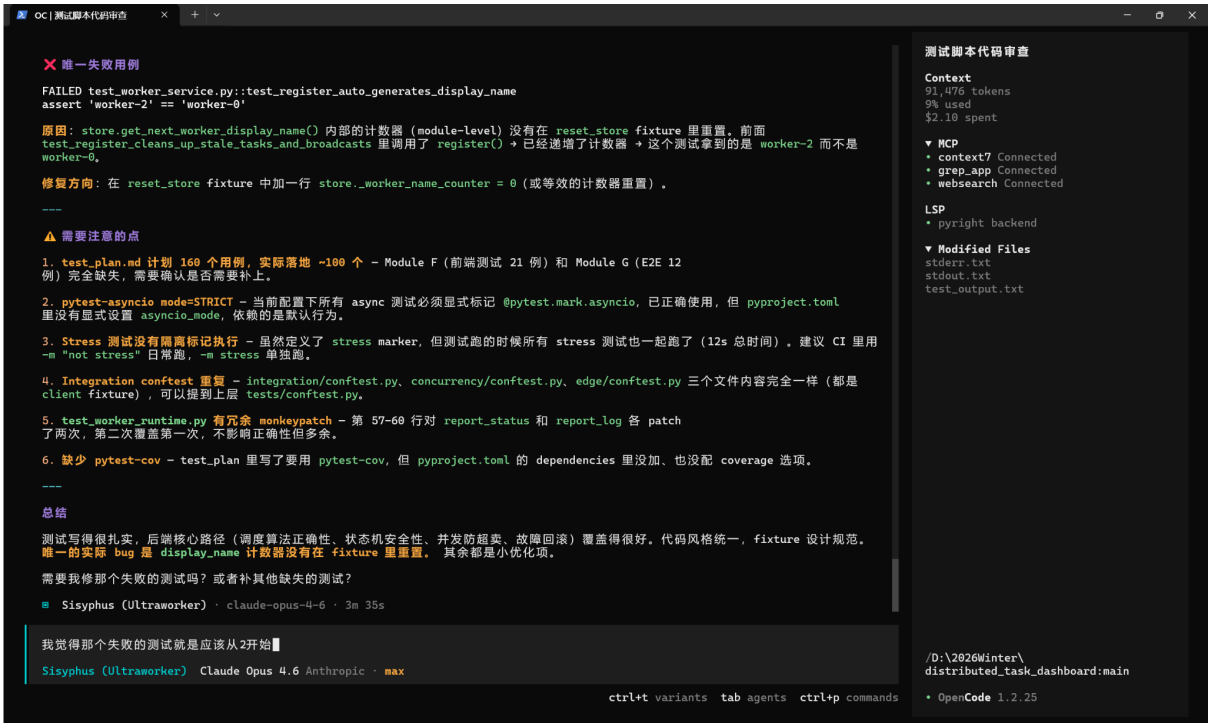
输入：

请调用前端agent实现 LogModal.vue 的日志滚动功能。
具体交互需求：

1. 默认情况下，收到新的 WebSocket 日志时，窗口要自动平滑滚动到最底部。
2. 关键体验：如果我用鼠标往上翻，去查看历史日志，必须立刻停止自动滚动（不能跟我抢滚动条）。当我手动滚回最底部时，再恢复自动滚动。
3. 性能要求：Worker 有时候一秒钟会狂发几百条日志。你必须处理好性能问题，绝对不能因为高频的 DOM 更新把浏览器卡死。

请修改代码，并在完成后简单给我解释一下，你是用什么技术方案来解决‘高频日志卡死浏览器’这个问题的？”

Agent输出：



输入：
当前开发状态已经基本完成，请你重新阅读docs中的文件，建立你对项目的理解，并针对项目生成一个完整的测试计划，需要包含edge cases，高并发，压力测试验证 Bin Packing 算法是否严格按照剩余资源分配，等极端场景的测试并把测试计划文档写在/docs目录中，完成后根据测试文档完成测试